

Type conversion

Mushfiqur Rahman Fakir

Lecturer

Dept. of CSE

Type conversions in expressions

- Automatic /implicit type conversion or type promotion
 - Explicit Type Conversion or Casting a value/type casting
-

Automatic type conversion

- If the operands are of different type the lower type is automatically converted to the higher type before the operation proceeds. The result is of higher type.
- Ex:
 - `int i,x;`
 - `float f;`
 - `double d;`
 - `long int l;`
- `x=l / i + i*f -d;`

Automatic type conversion

- All short and char are automatically converted to int, then,
- If either of the operand is of type long double, then others will be converted to long double and result will be long double.
- Else, if either of the operand is double, then others are converted to double.
- Else, if either of the operand is float, then others are converted to float.
- Else, if either of the operand is unsigned long int, then others will be converted to unsigned long int.
- Else, if one of the operand is long int, and the other is unsigned int, then
 - if a long int can represent all values of an unsigned int, the unsigned int is converted to long int.
 - otherwise, both operands are converted to unsigned long int.
- Else, if either operand is long int then other will be converted to long int.
- Else, if either operand is unsigned int then others will be converted to unsigned int.

Automatic type conversion

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
printf("%u %u", sizeof(unsigned int), sizeof(long int));
```

```
return 0;
```

```
}
```

 C:\Users\AUVI\Desktop\codeblocks\filee\bin\Debug\filee.exe

4 4

Process returned 0 (0x0) execution time : 1.192 s

Press any key to continue.

eds. The result is of the higher type. A typical type conversion sequence is shown in Fig. 3.7.

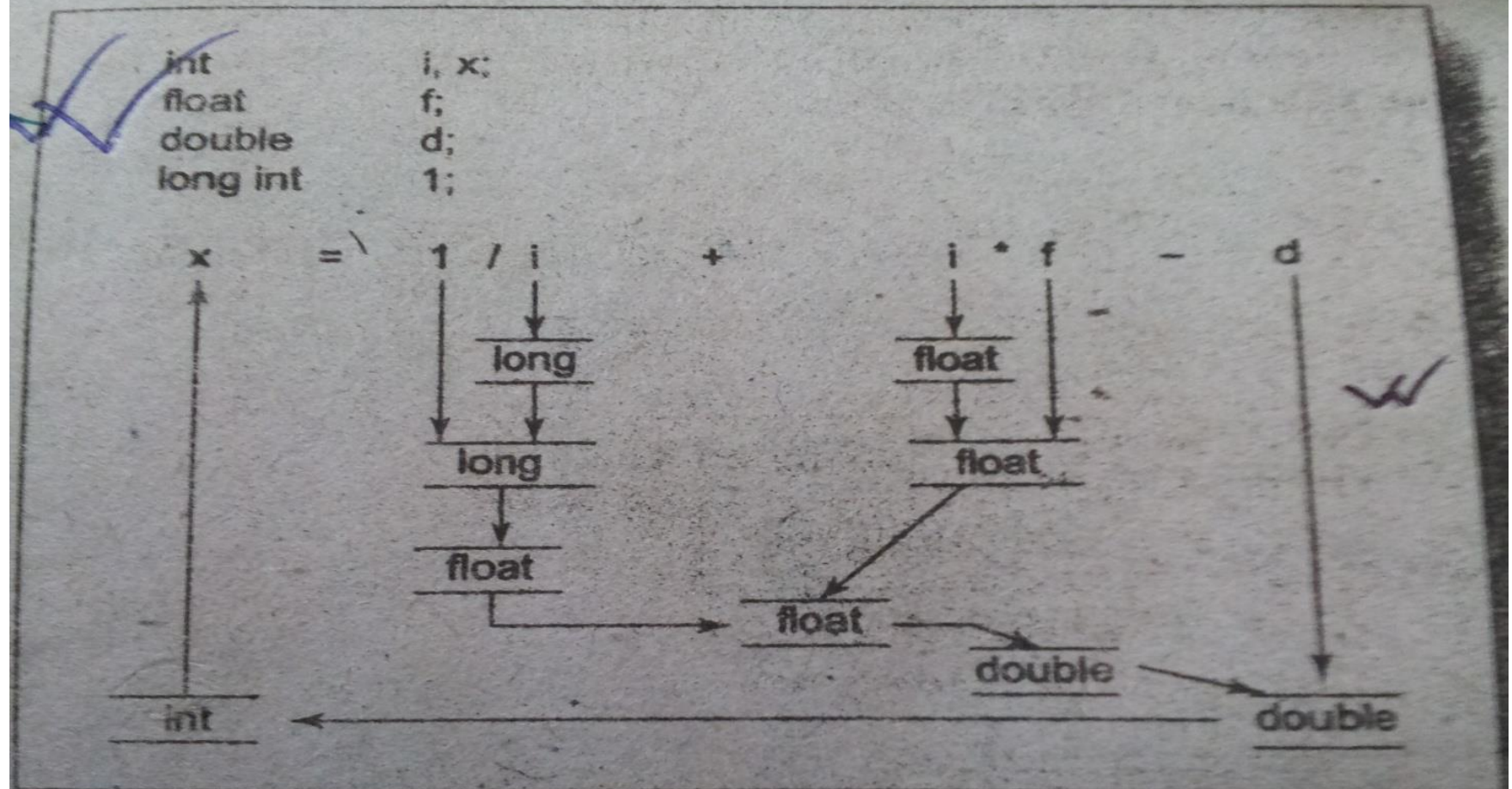


Fig. 3.7 Process of implicit type conversion

the sequence of rules that are applied while evaluating

Order of type

- The final result of an expression is converted to the type of the variable on the left of the assignment sign before assigning the value to it.

Order of type

70 | Programming in ANSI C

Conversion Hierarchy

Note that, C uses the rule that, in all expressions except assignments, any implicit type conversions are made from a lower size type to a higher size type as shown below:

Conversion Hierarchy

long double

double

float

unsigned long int

long int

unsigned int

int

short char

Note that some versions of C automatically convert all floating-point operands to double.

The final result of an expression is converted to the type of the variable on the left of the assignment sign before assigning the value.

Some problems during conversion

- float to int causes truncation of the fractional part.
 - double to float causes rounding of the digits.
 - long int to int causes dropping of the excess higher order bits.
-

Type Conversion in Assignment

```
main() {  
    float f=1234.5678;  
    int i;  
    i=f;  
    printf("%f %d", f, i);  
}
```

```
main() {  
    float f=10/3;  
    printf("%f", f);  
}
```

Type Cast

- Sometimes it is necessary to force a type conversion that is different from automatic conversion.
 - you want to use a variable in modulus operation, so you have declared it as an integer since modulus operation never accepts variables other than integers
 - But you may require using the variable in division as well which may produce a floating point result
 - You can use type cast this time
-

program

```
void main(){  
    int a=8;  
    int b=3;  
    int mod;  
    float result;  
    mod=a%b; ←  
    result=a/b; ←  
    printf("mod=%d",mod);  
    Printf("result=%f",result);  
}
```

program

```
void main(){  
    int a=8;  
    int b=3;  
    int mod;  
    float result;  
    mod=a%b; ←  
    result=(float)a/b; ←  
    printf("mod=%d",mod);  
    Printf("result=%f",result);  
}
```

Type Cast

```
float f=100.2;  
printf("%d", (int) f);
```

Type Cast

- you cannot cast a variable that is on the left side of an assignment statement.

```
int num;  
(float) num=1234.567;
```

```
int i = 7;  
float f = 5.5;  
char c = 'w'
```

ASCII Value

w = 119

0 = 48

i + f



12.5

float (double)

i + c



126

integer

i + c - '0'



78

integer

(i + c) - (2 * f /



123.8

float (double)

5)


```
4  int main()  
5  {  
6      int a = 10;  
7      float f = 5.5, sum;  
8  
9      sum = a + f;  
10  
11     printf("%f", sum);  
12  
13     return 0;  
14 }
```

■ Type conversion

Valid or Invalid?

```
6  
7  
8
```

```
int i, result;  
float f;
```

```
i = 7;  
f = 8.5;
```

```
result = (i + f) % 4;
```



- Type conversion 2

Valid or Invalid?

```
float num = 10.5;  
num % 2;
```



```
float num = 10.5;  
((int)num) % 2;
```



program

```
#include <stdio.h>
int main(){
    double x,d=5.0;
    int y;
    x=d*(x=2.5/d);
    printf("x=%lf\n",x);
    x=d*(y=(int)2.5 + 1.5);
    printf("x=%lf y=%d \n",x,y);
    return 0;
}
```

program

```
#include <stdio.h>
int main(){
    double x,d=5.0;
    int y;
    x=d*(x=2.5/d);
    printf("x=%lf\n",x);
    x=d*(y=(int)2.5 + 1.5);
    printf("x=%lf y=%d \n",x,y);
    return 0;
}
```

x=2.500000

x=15.000000 y=3

program

Explanation :

- The expression '`x=d*(x=2.5/d)`' is executed as below:-

```
x= d * ( x = 2.5 / d)
```

```
x= 5.0 * ( x = 2.5 / 5.0)
```

```
x= 5.0 * ( x = 0.5)
```

```
x= 2.5.
```

In `printf()`, '`%lf`' prints 2.5 in double format i.e **2.500000**.

- The expression '`x=d*(y=(int)2.5 + 1.5)`' is executed as below:-

```
x= d * ( y = (int) 2.5 + 1.5);
```

```
x=5.0 * ( y = 2 + 1.5) // 2.5 is typecasted to integer.
```

```
x=5.0 *( y = 3.5)
```

```
x=5.0 * 3.          // int y only stores integer part of 3.5 i.e 3.
```

```
x=15.0
```

In `printf()`, '`%lf`' prints the value x i.e **15.000000** and '`%d`' prints y i.e **3**.

	Operator	Associativity	Precedence
() [] . ->	Function call Array subscript Dot (Member of structure) Arrow (Member of structure)	Left-to-Right	Highest 14
! - - ++ -- & * (type) sizeof	Logical NOT One's-complement Unary minus (Negation) Increment Decrement Address-of Indirection Cast Sizeof	Right-to-Left	13
* / %	Multiplication Division Modulus (Remainder)	Left-to-Right	12
+	Addition	Left-to-Right	11
-	Subtraction		
<< >>	Left-shift Right-shift	Left-to-Right	10
<	Less than	Left-to-Right	8
<= > >=	Less than or equal to Greater than Greater than or equal to		
==	Equal to	Left-to-Right	8
!=	Not equal to		
&	Bitwise AND	Left-to-Right	7
^	Bitwise XOR	Left-to-Right	6
	Bitwise OR	Left-to-Right	5
&&	Logical AND	Left-to-Right	4
	Logical OR	Left-to-Right	3
? :	Conditional	Right-to-Left	2
=, += *=, etc.	Assignment operators	Right-to-Left	1
,	Comma	Left-to-Right	Lowest 0

Table 5.1: Precedence and Associativity Table

Precedence and Associativity

- `if(x==10+15 && y<10)`
- `a=b=5`

```
#include <stdio.h>
int main()
{
    int y;
    int z=y=5;
    printf("%d", z);
    return 0;
}
```



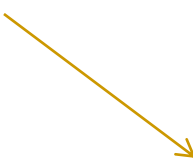
5

Program I

```
#include <stdio.h>
int main()
] {
int i = 1;
i=2+2*i++;
printf("%d",i);
return 0;
- }
```

Program I

```
#include <stdio.h>
int main()
{
    int i = 1;
    i=2+2*i++;
    printf("%d",i);
    return 0;
}
```



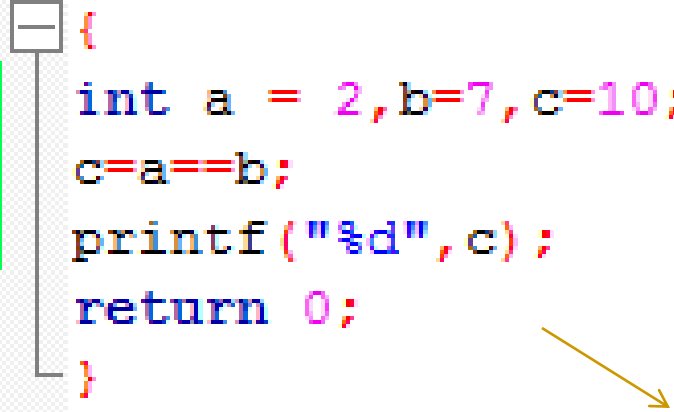
5

Program II

```
#include <stdio.h>
int main()
{
    int a = 2, b=7, c=10;
    c=a==b;
    printf("%d", c);
    return 0;
}
```

Program II

```
#include <stdio.h>
int main()
{
    int a = 2, b=7, c=10;
    c=a==b;
    printf("%d", c);
    return 0;
}
```



0

Program III

```
#include <stdio.h>
int main()
{
    int a=5,b=10;
    if(a=0)
        printf("true");
    else
        printf("false");
    return 0;
}
```

Program III

```
#include <stdio.h>
int main()
{
    int a=5,b=10;
    if(a=0)
        printf("true");
    else
        printf("false");
    return 0;
}
```

false

Program IV

```
#include <stdio.h>
int main()
{
    int y=2;
    int z=y+(y=10);
    printf("%d", z);
    return 0;
}
```

Program IV

```
#include <stdio.h>
int main()
{
    int y=2;
    int z=y+(y=10);
    printf("%d", z);
    return 0;
}
```



20

Program V

```
#include <stdio.h>
int main()
{
    int z;
    z=1,2,5;
    printf("%d",z);
    return 0;
}
```

Program V

```
#include <stdio.h>
int main()
{
    int z;
    z=1,2,5;
    printf("%d",z);
    return 0;
}
```

↓
1

References

- Web
 - <http://www.tutorialspoint.com/cprogramming/>
-